

LABSHEET - 8**HIMANI CHAUDHARY****BTECH(CSE) 3RD YEAR****CU240250967****SECTION - B****ROLL NO. - 24****Mastery in Data Analytics and Visualizations and Career Advancement (Technical)**

```
# EXPERIMENT NO. 8
# CUSTOMER CHURN PREDICTION USING DECISION TREE
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)
```

```
# 1. CREATE DATASET
# =====

np.random.seed(42)

n = 500

data = {
    "tenure": np.random.randint(1, 73, n),
    "MonthlyCharges": np.random.randint(20, 121, n),
    "TotalCharges": np.random.randint(100, 8000, n),
    "SeniorCitizen": np.random.randint(0, 2, n),
    "Contract": np.random.randint(0, 3, n),
    "InternetService": np.random.randint(0, 3, n)
}

df = pd.DataFrame(data)
print("Himani Chaudhary")
print("CU240250967")
```

Himani Chaudhary
CU240250967

```
# 2. CREATE CHURN COLUMN
# =====

df["Churn"] = (
    (df["tenure"] < 20) |
    (df["MonthlyCharges"] > 90) |
    (df["Contract"] == 0)
).astype(int)

print("\n=====")
print("CUSTOMER CHURN DATASET")
print("=====")

print(df.head())

print("\nDataset shape:")
print(df.shape)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CUSTOMER CHURN DATASET
=====
   tenure  MonthlyCharges  TotalCharges  SeniorCitizen  Contract  \
0       52             89          2304             1         2
1       15             50          2697             0         1
2       72             38          7708             1         2
3       61             80          1220             0         0
4       21             73          3168             0         0

   InternetService  Churn
0                 0      0
1                 1      1
2                 0      0
3                 0      1
4                 2      1

Dataset shape:
(500, 7)
Himani Chaudhary
CU240250967
```

```
# 3. CHECK MISSING VALUES
# =====

print("\nMissing values:")
print(df.isnull().sum())
print("Himani Chaudhary")
print("CU240250967")
```

```
Missing values:
tenure           0
MonthlyCharges   0
TotalCharges     0
SeniorCitizen    0
Contract         0
InternetService  0
Churn            0
dtype: int64
Himani Chaudhary
CU240250967
```

```
# 4. SEPARATE FEATURES AND TARGET
# =====

X = df.drop("Churn", axis=1)

y = df["Churn"]

print("\n=====")
print("FEATURES AND TARGET")
print("=====")

print("Features:", X.columns.tolist())
print("Features shape:", X.shape)
print("Target shape:", y.shape)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
FEATURES AND TARGET
=====
Features: ['tenure', 'MonthlyCharges', 'TotalCharges', 'SeniorCitizen', 'Contract', 'InternetService']
Features shape: (500, 6)
Target shape: (500,)
Himani Chaudhary
CU240250967
```

```
# 5. TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y)
```

```
)

print("\n=====")
print("TRAIN TEST SPLIT")
print("=====")

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
TRAIN TEST SPLIT
=====
Training data: (400, 6)
Testing data: (100, 6)
Himani Chaudhary
CU240250967
```

```
# 6. CREATE DECISION TREE
# =====

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)
print("Himani Chaudhary")
print("CU240250967")
```

Himani Chaudhary
CU240250967

```
# 7. TRAIN MODEL
# =====

model.fit(X_train, y_train)

print("\n=====")
print("MODEL TRAINED SUCCESSFULLY")
print("=====")
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
MODEL TRAINED SUCCESSFULLY
=====
Himani Chaudhary
CU240250967
```

```
# 8. PREDICTION
# =====

y_pred = model.predict(X_test)

print("\nFirst 20 predictions:")
print(y_pred[:20])
print("Himani Chaudhary")
print("CU240250967")
```

First 20 predictions:
[1 0 1 0 0 1 1 0 1 1 1 0 1 1 0 1 0 1 0]
Himani Chaudhary
CU240250967

```
# 9. CONFUSION MATRIX
# =====

cm = confusion_matrix(y_test, y_pred)

print("\n=====")
print("CONFUSION MATRIX")
print("=====")

print(cm)
```

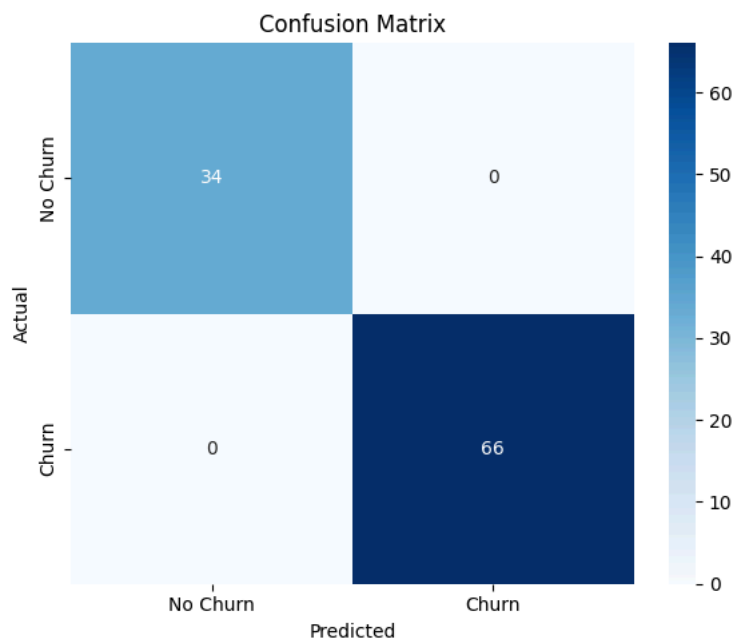
```
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CONFUSION MATRIX
=====
[[34  0]
 [ 0 66]]
```



Himani Chaudhary
CU240250967

```
# 10. CALCULATE PERFORMANCE
# =====

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\n=====")
```

```
print("MODEL PERFORMANCE")
print("=====")

print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall    :", round(recall * 100, 2), "%")
print("F1 Score  :", round(f1 * 100, 2), "%")
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
MODEL PERFORMANCE
=====
Accuracy : 100.0 %
Precision : 100.0 %
Recall    : 100.0 %
F1 Score  : 100.0 %
Himani Chaudhary
CU240250967
```

```
# 11. CLASSIFICATION REPORT
# =====

print("\n=====")
print("CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Churn", "Churn"],
        zero_division=0
    )
)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
No Churn	1.00	1.00	1.00	34
Churn	1.00	1.00	1.00	66
accuracy			1.00	100
macro avg	1.00	1.00	1.00	100
weighted avg	1.00	1.00	1.00	100

```
Himani Chaudhary
CU240250967
```

```
# 12. DISPLAY DECISION TREE
# =====

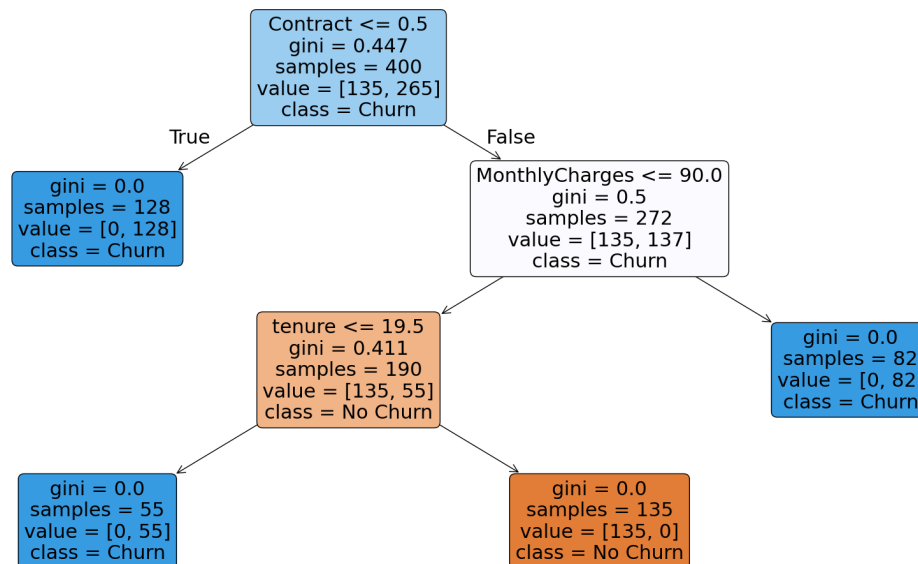
plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True
)

plt.title("Decision Tree - Customer Churn Prediction")

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")
```

Decision Tree - Customer Churn Prediction



Himani Chaudhary
CU240250967

```
# 13. FEATURE IMPORTANCE
# =====

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    "Importance",
    ascending=False
)

print("\n=====")
print("FEATURE IMPORTANCE")
print("=====")

print(importance)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
FEATURE IMPORTANCE
=====
   Feature  Importance
0    tenure    0.436941
1  MonthlyCharges  0.323325
4     Contract    0.239734
2  TotalCharges    0.000000
3  SeniorCitizen    0.000000
5  InternetService  0.000000
Himani Chaudhary
CU240250967
```

```
# 14. FEATURE IMPORTANCE GRAPH
# =====

plt.figure(figsize=(8, 5))

sns.barplot(
    x="Importance",
```

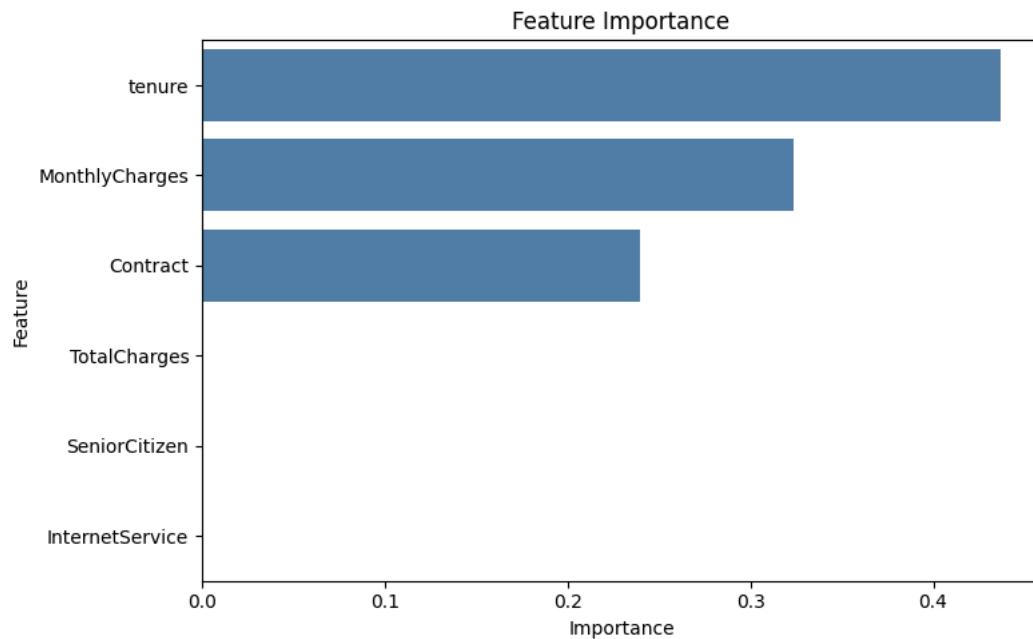
```

y="Feature",
data=importance,
color="steelblue"
)

plt.title("Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")

```



Himani Chaudhary
CU240250967

```

# 15. ACTUAL VS PREDICTED
# =====

result = pd.DataFrame({
    "Actual_Churn": y_test.values,
    "Predicted_Churn": y_pred
})

print("\n=====")
print("ACTUAL VS PREDICTED")
print("=====")

print(result.head(20))
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
ACTUAL VS PREDICTED
=====

```

	Actual_Churn	Predicted_Churn
0	1	1
1	0	0
2	1	1
3	0	0
4	0	0
5	1	1
6	1	1
7	0	0
8	1	1
9	1	1
10	1	1
11	1	1
12	0	0
13	1	1
14	1	1
15	0	0

```
16          1          1
17          0          0
18          1          1
19          0          0
Himani Chaudhary
CU240250967
```

```
# 16. SAVE RESULTS
# =====

result.to_csv(
    "churn_prediction_output.csv",
    index=False
)

importance.to_csv(
    "feature_importance.csv",
    index=False
)

df.to_csv(
    "customer_churn_dataset.csv",
    index=False
)

print("Himani Chaudhary")
print("CU240250967")
```

Himani Chaudhary
CU240250967

```
# 17. FINAL RESULT
# =====

print("\n=====")
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=====")

print("Student Name: HIMANI CHAUDHARY")

print("\nFinal Results:")

print("Accuracy  :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall     :", round(recall * 100, 2), "%")
print("F1 Score   :", round(f1 * 100, 2), "%")

print("\nFiles created:")
print("1. customer_churn_dataset.csv")
print("2. churn_prediction_output.csv")
print("3. feature_importance.csv")

print("\nDecision Tree Churn Prediction Completed!")
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
EXPERIMENT COMPLETED SUCCESSFULLY
=====
Student Name: HIMANI CHAUDHARY

Final Results:
Accuracy  : 100.0 %
Precision : 100.0 %
Recall    : 100.0 %
F1 Score  : 100.0 %

Files created:
1. customer_churn_dataset.csv
2. churn_prediction_output.csv
3. feature_importance.csv

Decision Tree Churn Prediction Completed!
Himani Chaudhary
CU240250967
```

